

BACKEND & AGENT INTERVIEW GUIDE

Agent / FastAPI 与简历技术栈专项面试指南

项目深挖版 | LangGraph · RAG · FastAPI · Java 后端

依据个人简历与当前项目代码整理 | 前端不作为个人主导经历 | 论文部分排除

王书文

专项复习版 · 2026 年 8 月

目录与使用方法

- 第一部分 面试定位、项目边界与 90 秒介绍
- 第二部分 Agent 服务整体架构与请求链路
- 第三部分 FastAPI 核心八股与项目落地
- 第四部分 Python 异步编程与工程基础
- 第五部分 LangChain、模型网关与提示词边界
- 第六部分 LangGraph 状态、节点、路由与持久化
- 第七部分 RAG 向量流水线、Chroma 与引用
- 第八部分 SSE 流式协议、取消与错误治理
- 第九部分 Redis checkpoint、MongoDB 冷恢复与异步归档
- 第十部分 并发、一致性、幂等与可靠性
- 第十一部分 Java、Spring Boot、MyBatis 与 MySQL
- 第十二部分 Redis、RabbitMQ 与微服务基础
- 第十三部分 简历技术栈追问与诚实回答边界
- 第十四部分 模拟面试、复习计划与自测清单
- 附录 官方资料与项目代码复习路径

复习顺序：先背熟 90 秒项目介绍，再按“FastAPI 请求层 → LangGraph 编排 → RAG 数据流 → SSE 与上下文 → Java 业务后端”顺序复盘。每个问题至少能回答：为什么、怎么做、失败怎么办、还有什么边界。

真实性原则：可以明确说前端由 AI 工具辅助生成、你主要负责后端与 Agent 链路。不要声称自己系统掌握 Vue/React；如果被问到前端，只说明你理解接口契约、SSE 事件和联调过程。

第一部分 面试定位、项目边界与 90 秒介绍

1.1 这份岗位中你的最佳定位

推荐定位：偏后端与 LLM 应用工程的实习候选人，能够用 Python/FastAPI 提供 Agent 微服务，使用 LangChain/LangGraph 编排 RAG 问答，并能与 Java/Spring Boot 业务系统完成接口集成。

岗位关注点	你的真实证据	面试表达
Python / FastAPI	独立 Agent 服务、请求校验、StreamingResponse、lifespan 资源管理	可主讲
LangChain / LangGraph	条件路由、查询重写、状态图、checkpoint、自定义流事件	可主讲
RAG	文档解析、分块、Embedding、Chroma 增量索引、TopK 与阈值、引用	可主讲
Java 后端	Spring Boot、MyBatis、MySQL CRUD 与 Agent 网关集成	保留为工程基础
Redis / 消息队列	checkpoint、会话锁、Redis Stream 归档；了解 RabbitMQ	区分已实现与了解
前端	能说明 SSE/API 契约与联调，页面代码主要由 AI 辅助	不作为主导经历

1.2 30 秒自我介绍

参考话术：您好，我是王书文，目前是延安大学电子信息专业硕士，方向偏计算机技术。我在信创智能客服项目中主要负责 Python Agent 服务和 RAG 知识库：用 FastAPI 提供内部流式接口，用 LangGraph 编排问题路由、查询改写、检索和生成节点，并使用 Chroma 保存维修手册向量索引。我也参与了 Spring Boot、MyBatis、MySQL 后台接口开发。我的优势是能把大模型调用做成有状态、可降级、可追踪的后端服务。

1.3 90 秒项目介绍

参考话术：项目是面向信创产品售后场景的智能客服系统。Java 服务负责用户、会话、工单、手册元数据和 MySQL 事务；Python Agent 服务负责模型编排与 RAG。一次请求由 Java 生成 requestId 和策略后调用 FastAPI 的 POST SSE 接口。Agent 先规范输入，再用模型判断 DIRECT 或 RAG；RAG 分支会结合历史改写查询，从 Java 发布清单增量同步维修手册到 Chroma，再执行 TopK 检索。检索到的引用先于正文 delta 发送，生成结果通过 meta、status、citation、delta、done 或 error 事件流返回。多轮上下文由 LangGraph checkpoint 保存到 Redis，Redis 热状态丢失后从 MongoDB 归档恢复；同一会话用锁避免并发写乱。我重点做的是让 RAG 链路不仅能跑，还能处理索引过期、模型失败、断连、取消和上下文恢复。

1.4 高频开场追问

你在项目中最核心的贡献是什么？

我最核心的贡献是 Agent 编排和向量知识库。具体包括 LangGraph 状态图、DIRECT/RAG 条件路由、查询改写、检索降级、流式事件，以及维修手册从发布清单到 Chroma 的增量同步。我参与了 Java 后台 CRUD，但不会把前端代码说成自己独立完成。

这个项目为什么需要拆成 Java 与 Python 两个服务？

Java 适合承载现有业务、权限、事务和 MySQL 数据；Python 的 LLM/RAG 生态迭代更快。拆分后双方通过稳定 DTO 和 SSE 协议协作，能独立部署、扩缩容和替换模型，同时要承担网络调用、契约兼容和分布式一致性的成本。

你最有技术含量的设计是什么？

一是 LangGraph 的受控条件工作流，不让模型随意决定所有步骤；二是 Java 发布清单驱动的 Chroma 增量索引，用 fingerprint 判断重建或删除；三是 Redis checkpoint 与 MongoDB 归档分层，兼顾在线延迟和长期恢复。

为什么没有把前端作为亮点？

我的主要投入在 Python Agent、RAG 和 Java 后端。前端由 AI 编程工具辅助生成，我参与的是接口契约、事件字段和联调验证，因此只把它当作系统边界知识。

项目已经生产可用了吗？

目前具备完整可运行链路和基础测试，但仍有明确生产化缺口，例如取消令牌还是进程内状态、归档仍用 Redis Stream 而非 RabbitMQ、内部鉴权需要升级为 TLS + HMAC + nonce、防重放，RAG 评测与监控也需要继续补齐。

第二部分 Agent 服务整体架构与请求链路

2.1 服务边界

组件	职责	关键技术
Java 业务服务	认证、会话、工单、手册元数据、MySQL 落库、对外 SSE	Spring Boot / MyBatis / MySQL / Redis
Python Agent 服务	模型编排、RAG、上下文、内部 SSE	FastAPI / LangChain / LangGraph
Chroma	保存可重建的维修手册向量切片	余弦空间 / 元数据过滤基础
Redis	LangGraph 热 checkpoint、会话锁、归档 Stream	TTL / Lock / Consumer Group
MongoDBStore	永久保存完成轮次并支持冷恢复	namespace + requestId 幂等键
模型网关	统一 Mock 与 OpenAI 兼容模型的流式/非流式调用	Protocol / ChatOpenAI

2.2 一次聊天请求的完整时序

1. Java 校验用户身份、会话和请求，生成 requestId，并构造 camelCase 的 Agent 策略。
2. Java 通过内部 Token 调用 POST /internal/ai/v1/chat/stream。
3. FastAPI 先用 Pydantic 校验请求，再读取活动角色；角色配置错误在建立 SSE 前返回 HTTP 500。
4. SSE 建立后先发送 meta，后续运行期异常只能转换为 error 终态事件。
5. AgentRuntime 以 userId + sessionId 生成 thread_id，获取同会话锁并注册 requestId。
6. Redis checkpoint 不存在时，从 MongoDBStore 读取最近完成轮次并写回 LangGraph 状态。

7. LangGraph 执行 input_guard、route_query，并根据 DIRECT/RAG 选择后续节点。
8. RAG 分支完成查询改写、清单同步、Chroma 检索和 citation 发送。
9. generate 节点裁剪上下文、流式调用模型并持续产生 delta。
10. 最终 AIMessage 写入 checkpoint，完成轮次先进入 Redis Stream；入队成功后 API 才发送 done。
11. Java 聚合答案和引用写入 MySQL，再生成浏览器可见的最终 done 事件。

面试关键：不要只画‘用户 → 模型’。面试官更关心数据归谁、终态由谁确认、模型失败怎么表示、上下文如何隔离、RAG 索引是否允许重建。

2.3 架构类八股

为什么 Python Agent 是内部服务，不直接暴露给浏览器？

用户认证、会话权限和 MySQL 事务属于 Java 业务域。只让 Java 调用 Agent 可以避免浏览器伪造 policy、knowledgeBases 或内部身份，也减少 Python 服务承担的安全边界。

为什么内部请求仍需要 Token？

内网不等于可信。Token 可以阻止误调用和部分横向移动，但固定 Token 只是最小保护；生产应使用 TLS、时间戳、nonce、请求体摘要、HMAC 和 Redis 防重放。

为什么 history 字段接收但不使用？

它是兼容旧 Java DTO 的过渡字段。正式上下文由 Redis/MongoDB 恢复，避免把 MySQL 中面向展示的消息误当作唯一模型历史，破坏上下文数据边界。

为什么 Java 才生成浏览器端 done？

Python 不知道 MySQL 最终消息主键。只有 Java 完成答案和引用落库后，才能对浏览器承诺本轮成功并返回可查询的消息 ID。

什么是派生数据？为什么 Chroma 可以删除重建？

原始手册文件和 Java MySQL 元数据是真相源，向量切片是根据它们计算得到的派生数据。只要版本、分块与 Embedding 配置可追溯，Chroma 可以安全重建。

这个架构的主要代价是什么？

跨进程网络延迟、SSE 协议复杂度、配置和密钥管理、分布式一致性以及更多运维组件。收益是业务与 AI 技术栈解耦、独立扩缩容、故障隔离和模型替换。

第三部分 FastAPI 核心八股与项目落地

3.1 项目中的 FastAPI 证据

- `create_app()` 应用工厂：支持注入 `Settings`、`ModelGateway` 和 `RagService`，方便测试替换。
- `lifespan` 异步上下文：统一创建和释放 `Redis`、`checkpointner`、`MongoDBStore` 等共享资源。
- `APIRouter + prefix`：健康检查和聊天接口统一挂载到 `/internal/ai/v1`。
- `Depends(verify_internal_token)`：把内部鉴权作为路由级依赖复用。
- `Pydantic v2`：请求校验、`camelCase` 别名、`extra=forbid`、字段 validator。
- `StreamingResponse`：异步生成 SSE；断连检测、终态事件和禁止代理缓冲响应头。
- `app.state`：保存运行时和配置，避免每次请求重建昂贵资源。

3.2 FastAPI 高频问答

FastAPI 为什么性能较好？

它基于 ASGI，常与 `Starlette`、`Uvicorn` 配合，能够用异步 I/O 高效处理大量等待型请求。性能优势主要来自事件循环和非阻塞 I/O，而不是所有 Python 代码都会自动并行。

ASGI 与 WSGI 的区别？

WSGI 主要面向同步请求响应；ASGI 原生支持 `async/await`、长连接和 `WebSocket` 等异步场景。LLM 流式输出持续时间长，ASGI 更合适。

`async def` 一定比 `def` 快吗？

不一定。`async def` 适合网络、数据库等 I/O 等待；CPU 密集任务会阻塞事件循环，需放到线程池、进程池或独立 worker。本项目把 `Chroma` 的同步调用放到 `asyncio.to_thread`。

FastAPI 如何做请求参数校验?

通过类型注解和 Pydantic 模型生成校验规则。项目对 requestId、sessionId、userId 设置 gt=0, 对 message 设置长度与空白校验, 对多余字段使用 extra=forbid。

为什么 Python 内部 snake_case、接口使用 camelCase?

Python 保持语言习惯, Pydantic alias_generator=to_camel 负责序列化/反序列化, 与 Java record 和前端契约对齐, 减少手工字段映射。

Depends 的价值是什么?

把鉴权、数据库会话或公共校验声明为依赖, 由框架解析和注入。项目把 X-Internal-Token 校验放在整个 chat router 上, 避免每个接口重复调用。

lifespan 用来解决什么问题?

在应用开始接收请求前创建共享资源, 在关闭时释放。项目在 lifespan 中创建 Redis checkpointer、通用 Redis 客户端和 MongoDBStore, 既避免每请求建连接, 也保证异常退出时按上下文顺序清理。

为什么使用应用工厂而不是模块级 app?

应用工厂允许按环境注入配置和替身依赖, 测试可使用 InMemorySaver 与 MockModel, 不访问真实 Redis/MongoDB; 也便于创建不同实例。

app.state 适合放什么?

适合应用生命周期内共享的客户端、运行时或配置, 不适合存用户级可变状态。项目把 AgentRuntime 和 Settings 放在 app.state, 用户上下文仍进入 LangGraph/Redis。

StreamingResponse 的 iterable 可以是什么?

可以是同步或异步迭代器。对于模型流, 本项目使用 AsyncIterator[str], 每次 yield 一个完整 SSE 帧, 避免等待答案全部生成。

为什么 SSE 建立后不能再返回 HTTP 500?

响应头已经发送, HTTP 状态无法修改。运行期错误必须编码为协议内的 error 事件, 并保证是终态; 建立连接前的角色配置或请求校验错误仍可使用正常 HTTP 状态。

如何检测客户端断开？

在事件生成循环中调用 `request.is_disconnected()`。发现断开后请求 `runtime.cancel(requestId)` 并停止继续消费模型流，减少无意义计算。

为什么设置 X-Accel-Buffering: no？

某些反向代理会缓冲小块响应，导致前端看不到实时 `delta`。该响应头配合 `no-cache`、`no-transform` 和代理配置，尽量禁止缓冲。

HTTPException 与领域异常如何分工？

`HTTPException` 用于建立响应前且与 HTTP 语义直接相关的问题；运行时内部抛出领域异常，由 SSE 路由映射成稳定 `error code`，避免把供应商异常原文泄漏出去。

如何做 FastAPI 测试？

使用 `TestClient` 或异步客户端启动 `lifespan`，注入 `MockModel/InMemorySaver`，验证状态码、校验错误、SSE 事件顺序、鉴权与取消幂等。外部 `Redis/MongoDB` 应通过配置关闭或替换。

如何部署 FastAPI？

一般由 `Uvicorn` 运行，前面配置反向代理和 TLS。多 `worker` 时要确保取消、锁和上下文不依赖单进程内存；还需设置超时、连接池、健康检查、日志和优雅关闭。

多 worker 对本项目有什么风险？

`CancellationRegistry` 目前是进程内字典，取消请求若落到另一个 `worker` 会失效。上线多实例前应把活动运行和取消标记改成 `Redis` 等共享状态。

为什么 SecretStr 不能代替密钥管理？

`SecretStr` 只减少 `repr` 或日志误打印，并不加密内存或配置文件。生产密钥仍应来自环境变量、`Secret Manager` 或编排平台，并限制权限与轮换。

第四部分 Python 异步编程与工程基础

4.1 异步与并发

协程是什么？

协程是可暂停和恢复的执行单元。async def 创建协程函数，调用后得到协程对象，await 在等待 I/O 时把控制权交还事件循环。

事件循环做什么？

它调度就绪协程、处理 I/O 事件和计时器。一个协程执行阻塞 CPU 或同步 I/O 时，会拖慢同一循环上的其他请求。

asyncio.to_thread 适合什么？

适合把不可避免的同步阻塞调用放到线程池，避免阻塞事件循环。本项目用它包装 Chroma 同步、检索和索引操作；它不让纯 Python CPU 任务绕过 GIL。

asyncio.Lock 与 Redis Lock 的区别？

asyncio.Lock 只在单进程事件循环内有效；Redis Lock 可跨进程/实例。本项目持久化模式用 Redis 会话锁，测试模式用进程内锁保持相同行为。

为什么锁要按 userId + sessionId 粒度？

同一会话的两个请求会并发更新同一 LangGraph thread，可能破坏消息顺序；不同会话互不影响，不应使用全局锁降低并发。

什么是 async context manager？

实现异步进入和退出逻辑的上下文管理器，常由 @asynccontextmanager + yield 编写。本项目用于 FastAPI lifespan 与 Redis 客户端生命周期。

异步生成器有什么价值？

async for 可以边等待边消费数据，async yield 可以逐块输出。模型 token 与 SSE 都天然适合异步生成器，内存占用和首包延迟更好。

如何传播异步取消？

要在可控边界检查取消标记、关闭上游流并在 finally 清理注册表和锁。本项目在图流和模型 delta 循环中检查 requestId。

4.2 类型、模型与工程习惯

TypedDict、dataclass 和 Pydantic BaseModel 怎么选？

TypedDict 适合描述运行期字典状态，如 LangGraph AgentState；dataclass 适合内部不可变运行上下文；Pydantic 适合外部输入输出，需要校验、别名和序列化。

Protocol 有什么作用？

它支持结构化子类型，只要对象实现约定方法即可被视为接口。本项目用 ModelGateway Protocol 隔离 LangGraph 与具体模型供应商。

为什么 AgentRunContext 使用 frozen dataclass？

这些字段是本次运行参数，不应被节点修改，也不需要写入长期 checkpoint。frozen 和 slots 能明确不可变边界并减少意外属性。

异常应该捕获到什么粒度？

只捕获能处理或能转换语义的异常。路由与查询改写失败可以降级；模型主生成失败必须终止；通用 Exception 只在 API 边界兜底并记录堆栈。

lru_cache 用在 Settings 上有什么影响？

同一进程共享一个配置实例，减少重复解析 .env。测试若需要不同配置，应直接注入 Settings 或清理缓存，不能依赖全局环境残留。

为什么使用 Ruff 和 pytest？

Ruff 统一导入、语法和常见 bug 检查；pytest 验证协议与边界。项目测试覆盖 SSE 顺序、空输入、requestId 重用、RAG 降级、索引增删改和上下文恢复。

什么是依赖注入？项目如何体现？

对象不在业务内部硬编码创建，而由外部传入。create_app 可注入 Settings、ModelGateway、RagService；AgentRuntime 依赖 checkpointer 和 archive 协议，测试可替换。

GIL 对本项目有什么影响？

网络 I/O 可通过异步提高并发；大量 Python CPU 计算仍受 GIL 影响。文档解析或向量本身主要在原生库/外部服务，但重 CPU 任务更适合进程池或独立任务服务。

第五部分 LangChain、模型网关与提示词边界

5.1 项目中真正使用了哪些 LangChain 能力

- ChatOpenAI：调用 OpenAI 或兼容协议的模型服务，支持 astream 与 ainvoke。
- BaseMessage / HumanMessage / AIMessage / SystemMessage：统一消息对象。
- trim_messages：接近因策略裁剪上下文，并保留 system 与当前用户消息。
- OpenAIEmbeddings：调用阿里云兼容接口的 text-embedding-v3。
- Document Loaders 与 RecursiveCharacterTextSplitter：加载和切分 PDF、DOCX、TXT、MD。
- langchain-chroma：以 Chroma 作为向量存储。

5.2 高频问答

LangChain 与 LangGraph 的区别？

LangChain 提供模型、消息、Embedding、Retriever 等组件；LangGraph 用于显式定义有状态工作流、分支、持久化和恢复。项目用 LangChain 组件，用 LangGraph 组织执行。

为什么还要定义 ModelGateway？

直接在节点里调用 ChatOpenAI 会让业务逻辑绑定供应商和 SDK。Gateway 只暴露 stream/complete，Mock 与真实模型共享接口，方便测试和切换私有模型。

路由为什么用非流式 complete？

路由输出只有 DIRECT/RAG，流式没有用户价值，反而增加协议复杂度；限制 16 个输出 token 也能控制延迟和成本。

为什么生成使用流式 stream?

主回答较长，流式可以降低用户感知首字延迟，并把 delta 直接交给 SSE。

trim_messages 的目的是什么?

checkpoint 可以保存较长历史，但模型有上下文窗口。生成前按最大输入 token 裁剪最近消息，同时检查当前用户消息是否仍存在，防止输入本身超限。

系统提示词应该放在哪里?

角色提示词作为本次运行上下文快照传入，不持久化进对话 messages；RAG 证据只在当前生成时拼到 system prompt，避免污染长期历史。

如何防止提示词注入?

不能只依靠一句‘忽略用户指令’。要限制工具和数据权限、把检索内容视为不可信数据、分离系统与证据、校验输出和引用，并在网关层做审计。当前项目主要通过受控节点与无工具模式降低风险。

模型输出为空怎么办?

generate 聚合文本后检查非空，output_validate 再检查最后一条消息是有效 AIMessage；失败转换为 OUTPUT_VALIDATION_FAILED，而不是写入成功结果。

为什么 toolsEnabled=false?

当前 RAG 节点是确定性流程，不需要让模型自主选择工具。关闭工具调用可以降低不可控行为、权限风险和调试成本。

Mock 模型有什么意义?

它不依赖密钥和网络，能稳定测试 FastAPI、LangGraph、SSE、上下文和取消链路；但不能代表真实模型质量，仍需单独做线上模型评测。

第六部分 LangGraph 状态、节点、路由与持久化

6.1 当前状态图

1. START → input_guard: 规范化用户输入并清空本轮临时 RAG 状态。

2. input_guard → route_query: 判断 DIRECT 或 RAG; 格式异常与调用失败默认 RAG。
3. DIRECT → generate: 跳过检索, 直接按角色生成。
4. RAG → rewrite_query: 把多轮追问改写成独立检索查询。
5. rewrite_query → sync_manual_index: 以 Java 发布清单同步 Chroma。
6. sync_manual_index → retrieve: TopK 检索、阈值过滤并发送 citation。
7. retrieve → generate: 组合角色、证据和历史, 流式生成。
8. generate → output_validate → END: 确保终态存在有效 AIMessage。

6.2 高频问答

为什么使用图而不是一条 Chain?

流程存在 DIRECT/RAG 分支、临时状态、检查点、恢复和多个流事件。图结构把控制流显式化, 节点可单测、失败可定位, 后续增加审核或工具节点也更清楚。

LangGraph 的 State、Node、Edge 分别是什么?

State 是共享快照; Node 接收状态并返回增量更新; Edge 决定下一个节点。条件边根据 route 在 DIRECT 和 RAG 之间分支。

MessagesState 的 reducer 做什么?

messages 使用 add_messages reducer: 新 ID 追加、相同 ID 替换。项目因此给每次真实运行生成 UUID 消息前缀, 避免开发环境 requestId 重用覆盖旧消息。

为什么 route 等字段没有 reducer?

它们只属于当前一轮, input_guard 每次重置, 后续节点覆盖保存; 不应该像 messages 一样跨轮累积。

为什么 AgentRunContext 不写入 State?

requestId、模型路由、系统提示词和 token 上限是本次调用参数, 不应成为长期对话历史。context_schema 可以让节点访问它们, 同时避免 checkpoint 污染。

路由失败为什么默认 RAG?

在客服手册场景，漏检会让模型对企业事实直接臆测；多检一次成本可控。默认 RAG 是偏向事实安全的失败策略。

查询重写失败为什么回退原问题?

改写是增强项，不是主链路硬依赖。用原问题检索仍有机会命中，避免辅助模型故障让整轮失败。

索引同步失败为什么不查询旧 Chroma?

旧索引可能包含已归档、禁用或过期手册。宁可明确降级为通用回答，也不能用无法确认新鲜度的企业知识。

stream_mode=[custom, values] 有什么用?

custom 接收节点主动写出的 status、citation、delta；values 接收每一步完整状态，运行结束后用于取得最终 AIMessage。

checkpoint 与 Store 有什么区别?

checkpoint 保存一个 thread 的执行状态，适合短期恢复和多轮上下文；Store 保存跨运行或长期数据。项目用 Redis checkpoint 做热状态，用 MongoDBStore 归档完成轮次。

thread_id 为什么包含 userId?

仅用 sessionId 可能在不同用户之间碰撞。user:{userId}:session:{sessionId} 明确租户隔离边界。

Shallow checkpoint 是什么考虑?

项目使用 AsyncShallowRedisSaver 以减少保留的历史 checkpoint 版本，在线场景更关注最新状态而非完整时间旅行；具体取舍要结合恢复与审计需求。

如何从冷存储恢复?

运行前先 aget_tuple 检查 Redis；未命中时从 MongoDBStore 取完整会话、显式排序、截取最近 N 轮，再通过 graph.aupdate_state 让 LangGraph 自己生成合法 checkpoint。

为什么不能直接拼 Redis checkpoint key?

checkpoint 是 LangGraph 的内部存储格式，手工拼 key 容易与版本、索引和序列化规则不兼容。应调用 checkpointer 或 graph API。

如何增加人工审核节点?

在生成或高风险动作前增加 review 节点, 通过条件边决定继续、重试或结束, 并使用 checkpointer + interrupt 保存暂停状态。当前客服问答没有启用该功能。

图是否等于 Agent?

不完全等同。图是一种可控编排方式; 是否称为 Agent 取决于是否包含模型决策、状态和行动。当前路由由模型辅助, 但 RAG 执行是确定性的, 不是完全自治 Agent。

第七部分 RAG 向量流水线、Chroma 与引用

7.1 数据流水线

1. Java 管理端是唯一上传和发布入口, MySQL 元数据与共享目录原文件是真相源。
2. Agent 通过带内部 Token 的 HTTP 请求拉取已发布手册清单。
3. 清单项使用 sha256 + resourceVersion 形成 fingerprint。
4. Agent 读取 Chroma 元数据: 新增/变化文档重建, 归档/禁用文档删除, 未变化跳过。
5. PDF、DOCX、TXT、MD 使用对应 Loader 解析, 并补充 sourceId、documentId、title、page。
6. RecursiveCharacterTextSplitter 按 800 字符、120 重叠切分。
7. text-embedding-v3 生成 1024 维稠密向量, Chroma 以 cosine 空间持久化。
8. 查询时取 Top 5, 并过滤低于 0.35 的相关度结果。
9. citation 在首个 delta 前发送, 答案提示词要求事实只能来自检索证据。

7.2 高频问答

RAG 的基本流程是什么?

离线/准实时侧完成加载、清洗、分块、Embedding 和索引; 在线侧完成查询理解、查询向量、相似度检索、可选重排、上下文拼接、生成与引用。

为什么分块？

整篇文档可能超过上下文且主题混杂。分块让检索定位更细，减少无关 token；过小会丢上下文，过大又降低精度和增加成本。

chunk overlap 的作用？

保留跨边界语义，避免关键步骤恰好被切断；但重叠太大会增加向量数量、存储、重复召回和成本。

为什么使用 fingerprint？

用内容哈希与资源版本判断索引是否仍对应真相源。这样只处理变化文档，也能在归档时删除旧切片。

为什么不直接扫描共享目录？

目录中可能有未发布、已禁用或残留文件，缺少业务状态。Java 发布清单才包含权限与生命周期语义。

Embedding 是什么？

把文本映射为稠密向量，使语义相近文本在向量空间更接近。它适合语义检索，但不天然保证精确关键词和数值匹配。

余弦相似度是什么？

衡量两个向量方向的接近程度，常用于文本 Embedding。Chroma 的 `hnsw:space=cosine` 指定近邻索引使用余弦距离语义。

TopK 越大越好吗？

不是。K 大提高召回但引入噪声、上下文成本和冲突证据；K 小可能漏掉答案。需要结合数据集、重排和生成质量评测。

阈值如何确定？

不能只凭感觉。应构造带标准答案和相关文档的查询集，观察召回率、精确率、无答案识别和最终生成质量，再选择阈值。0.35 是当前配置起点。

稠密检索的缺点？

对精确型号、错误码、数字和稀有词可能不如 BM25。生产可考虑稠密 + 稀疏混合检索、元数据过滤和 reranker。

为什么当前没有 reranker?

当前先实现可运行且可解释的基础链路，TopK 较小。reranker 能提升排序，但增加模型、延迟和成本，需要评测证明收益后再引入。

如何避免引用与答案不一致?

citation 使用本轮检索结果快照，生成 prompt 明确只依据证据；Java 保存同一轮引用。更严格时可做句子级引用校验、答案支持度评测和生成后验证。

没有检索结果怎么办?

明确说明未找到匹配手册，可以给通用建议但不得编造企业来源。知识库故障与正常无命中也要区分状态。

为什么引用先于 delta?

前端/Java 可以先建立来源上下文，并确保正文开始前已取得可落库引用；即使后续生成失败，也能清晰区分检索阶段与生成阶段。

索引同步时单个文件解析失败怎么办?

记录 documentId 和异常，跳过该文件但继续同步其余手册；该文档保持无切片，下次请求仍可重试。不能让一个坏文件阻断整个知识库。

为什么同步要加进程内锁?

避免同一 Agent 进程的多个 RAG 请求同时删除和重建同一 Chroma 集合。多实例部署时还需外部协调或独立索引服务。

路径穿越如何防护?

object_key 与根目录拼接后 resolve，并验证解析路径的 parent 等于指定根目录且文件存在，拒绝 ../ 越界。更通用时还应支持安全子目录判断。

RAG 如何评测?

分层评测：解析/分块正确性、检索 Recall@K/MRR、引用准确率、答案忠实度、无答案拒答、延迟与成本。还要保留真实客服难例做回归集。

第八部分 SSE 流式协议、取消与错误治理

8.1 事件协议

事件	用途	是否终态
meta	request/session/role 等本轮元信息	否
status	safety、intent、retrieval、generation 阶段提示	否
citation	检索来源快照	否
delta	模型增量文本	否
usage	模型用量信息，当前预留	否
done	成功完成，由 Python 内部完成后再由 Java 生成外部终态	是
error	稳定错误码、消息与 retryable	是

8.2 高频问答

SSE 与 WebSocket 的区别？

SSE 是服务端到客户端单向事件流，基于 HTTP、文本协议、实现简单；WebSocket 双向、适合高频交互。聊天输入仍走 POST、输出单向流时 SSE 足够。

为什么使用 POST SSE 而不是 EventSource？

请求体包含 message 与 policy，且需要内部 Token；浏览器原生 EventSource 主要发 GET 且自定义请求头受限。Java/Python 可以用普通 HTTP 客户端消费 POST 流。

SSE 帧为什么以空行结束？

协议用空行分隔事件。项目输出 event、id、data，每帧最后是两个换行；data 是统一 JSON envelope。

sequence 有什么用？

每个 request 内严格递增，便于检测丢帧、乱序和未来断线续传；不能把数据库 ID 当作流顺序。

为什么需要明确终态？

流连接关闭可能来自成功、网络断开或服务异常。done/error 让调用方知道本轮业务结果；如果连接无终态中断，应按异常恢复。

取消与客户端断开有什么区别？

用户主动取消会调用 cancel 接口；断开是传输层事实。两者都应停止上游模型，但取消接口还需要等待返回是否命中活动运行。

取消为什么只能尽力而为？

只有在模型流的可中断边界检查标记才能停止；供应商请求可能已经产生费用，底层 SDK 也未必立即取消。需要关闭 HTTP 流、设置超时并记录终态。

错误码为什么不能直接使用异常类名？

异常类和供应商文本是内部实现，可能变化或泄密。稳定 code 供 Java/前端分支处理，message 面向用户，retryable 表示重试建议。

哪些错误可重试？

模型暂时不可用、上下文存储暂时失败、会话已有运行通常可重试；输入过大、用户取消不应自动重试。重试还要结合幂等键和退避。

如何防止代理缓冲？

响应头设置 no-cache/no-transform、X-Accel-Buffering:no，并在 Nginx 等代理关闭 buffering；同时每个 delta 要及时 flush，必要时发送 heartbeat。

前端代码不是你写的，SSE 还需要会到什么程度？

需要能解释协议、事件顺序、终态、错误恢复和联调抓包；不必声称熟练 Vue，只要能证明你设计并验证了后端契约。

第九部分 Redis checkpoint、MongoDB 冷恢复与异步归档

9.1 分层设计

层	数据	目的
Redis checkpoint	LangGraph 最新 thread 状态, TTL 7 天	低延迟多轮上下文
Redis Stream	完成轮次归档事件	解耦在线请求与永久写入
MongoDBStore	永久完成轮次	Redis 过期后的冷恢复
MySQL	用户可见会话、消息、引用、业务事务	业务真相与查询

9.2 高频问答

为什么 Redis 适合热上下文？

内存访问延迟低、支持 TTL 和分布式锁，适合频繁读取的最近会话状态；但成本高且可能过期，不适合作为唯一永久存储。

为什么还需要 MongoDB？

Redis checkpoint 过期或丢失后需要恢复。MongoDBStore 永久保存完成轮次，模式相对灵活，按用户/会话 namespace 隔离。

为什么不直接从 MySQL chat_message 恢复？

MySQL 消息是业务展示真相，可能包含状态、编辑或与 Agent checkpoint 不一致。项目明确让 Agent 上下文由自己的 Redis/Mongo 体系维护，避免跨域耦合。

冷恢复为什么先全取再排序截断？

MongoDBStore 在无语义查询时不保证业务顺序；如果先 limit 可能截到任意旧记录。先取、按 created_at 和 requestId 排序，再保留最近 N 轮。

Redis Stream 是什么？

一种持久化追加日志结构，支持 consumer group、Pending Entries List、XACK 与消息接管，适合至少一次事件消费。

为什么写入 Mongo 后才能 XACK?

ACK 表示消息已成功处理。提前 ACK 后 Mongo 写失败会永久丢失归档；写入成功再 ACK，失败则留在 Pending 供重试。

至少一次投递如何实现幂等?

MongoDBStore 的 key 使用 requestId，同一轮重复投递会覆盖同一文档，而不是新增重复轮次。幂等键必须在业务上稳定唯一。

消费者崩溃后消息怎么办?

消息留在 Pending。健康消费者定期用 XAUTOCLAIM 接管超过 idle 时间的消息，写入成功后再 ACK。

为什么不对 Stream 直接 MAXLEN 裁剪?

粗暴裁剪可能删除仍在 Pending 的消息体，留下无法恢复的引用。应先设计安全保留策略、监控 lag/pending，再清理已确认数据。

Redis checkpoint 为什么必须 database 0?

当前 LangGraph Redis Saver 依赖 Redis Search 创建索引，而 Redis Search 不支持在非 0 逻辑数据库创建索引；配置校验提前拒绝错误 URL。

TTL 过期会丢对话吗?

热 checkpoint 会过期，但完成轮次已异步归档到 MongoDB；下一次请求可冷恢复。归档通道失败时本轮不会发送成功 done，以避免假成功。

第十部分 并发、一致性、幂等与可靠性

同一会话为什么不能并发运行?

两个请求会读取相同 checkpoint 并交错追加消息，导致上下文顺序和最终状态不确定。项目用非阻塞会话锁让第二个请求立即得到可重试错误。

为什么不用数据库事务跨 Java 与 Python?

分布式服务无法共享一个本地事务。应通过状态机、幂等、可重试事件和最终一致性协调。Java 只在自身 MySQL 内保证事务。

done 与持久化如何排序？

Python 先完成 checkpoint 和归档事件入队，再发内部 done；Java 聚合答案和引用写 MySQL 成功后才发浏览器 done。终态应尽量代表可查询结果。

requestId 的作用有哪些？

贯穿 Java 业务请求、Agent 取消、SSE envelope、归档幂等和日志关联。它是业务相关 ID，但 LangGraph 消息 ID 还额外加入 UUID 防止重建环境碰撞。

幂等接口如何设计？

选择稳定幂等键，记录处理结果或使写入可覆盖；重复请求返回已有结果，不能重复创建消息、扣费或发送副作用。取消接口返回 NOT_RUNNING 也是幂等语义。

超时应该设置在哪里？

浏览器/Java、Java→Python、Python→模型/清单服务都应有分层超时；还要区分连接、读取和整体请求时间，并让取消和清理在超时后生效。

降级和熔断有什么区别？

降级是用较弱能力继续，例如 RAG 不可用时给通用回答并声明未核对手册；熔断是在依赖连续失败时暂时拒绝调用，防止级联故障。当前项目实现了节点级降级，未完整实现熔断器。

重试有什么风险？

可能放大故障、重复消费或重复生成成本。必须只重试瞬时错误，采用指数退避与抖动，并保证写操作幂等。流式请求中途重试还要处理已发送部分。

如何做可观测性？

使用 requestId/threadId 关联结构化日志，记录节点耗时、路由结果、检索数量、首 token 延迟、总耗时、错误码和依赖状态；不能记录密钥或完整敏感 Prompt。

健康检查应该检查什么？

基础 liveness 只判断进程；readiness 可检查必要配置和关键连接。不要每次健康检查都执行昂贵模型调用，可以单独提供深度诊断。

如何避免雪崩？

设置并发上限、队列、超时、熔断、限流与缓存；对模型和 Embedding 分别限流；RAG 同步最好从每请求路径迁移到独立任务或版本发布事件。

当前最大的可靠性缺口是什么？

多实例取消未共享、Chroma 同步锁仅进程内、Redis Stream 未设置成熟保留策略、缺少完整 RAG 评测与指标、内部鉴权仍是固定 Token。

第十一部分 Java、Spring Boot、MyBatis 与 MySQL

11.1 Java 基础

ArrayList 与 LinkedList 的区别？

ArrayList 基于动态数组，随机访问快、尾部追加均摊 $O(1)$ ；LinkedList 节点分散、按下标访问 $O(n)$ ，在已定位节点时插删方便。多数业务场景优先 ArrayList。

HashMap 原理？

通过 hash 定位桶，冲突使用链表/红黑树结构；容量通常按 2 的幂扩容。它非线程安全，键的 equals/hashCode 必须一致。

== 与 equals 的区别？

基本类型 == 比值，对象 == 比引用；equals 可由类定义逻辑相等。重写 equals 通常必须同步重写 hashCode。

反射是什么？

运行时读取类、字段、方法并动态调用。Spring 的依赖注入、代理和注解处理广泛使用反射，但它降低静态可读性并有额外开销。

受检异常与非受检异常？

受检异常编译期要求处理，适合调用方可恢复的问题；RuntimeException 表示编程错误或不希望层层声明的问题。业务中应转换为稳定领域异常。

线程安全怎么理解？

多个线程并发访问共享状态时仍保持正确结果。常见手段有不可变对象、线程封闭、锁、原子类、并发容器和消息串行化。

11.2 Spring Boot 与事务

Spring IoC 是什么？

对象的创建和依赖关系由容器管理，业务类面向接口或构造器依赖，降低耦合并便于测试。

AOP 有哪些用途？

将日志、事务、权限、监控等横切逻辑从业务代码抽离，通过代理在方法前后织入。

@Transactional 为什么可能失效？

常见原因包括同类内部自调用绕过代理、方法非 public、异常被吞、抛出不触发回滚的异常、对象不是 Spring Bean 或跨线程执行。

事务传播 REQUIRED 与 REQUIRES_NEW？

REQUIRED 有事务就加入，没有就新建；REQUIRES_NEW 暂停外层并新建独立事务。后者适合必须独立提交的操作，但要注意连接数和一致性。

Spring Boot 自动配置原理？

根据 classpath、配置属性和已有 Bean 条件化注册默认组件，开发者可以通过自定义 Bean 或配置覆盖。

Controller、Service、Mapper 如何分层？

Controller 处理协议和校验；Service 组织业务规则与事务；Mapper 负责持久化。跨层不要泄漏过多实现细节。

Java 调用 Python Agent 要注意什么？

连接池、超时、流式读取、内部鉴权、DTO 版本、错误码映射、取消传播、断连清理和 requestId 日志关联。

11.3 MyBatis 与 MySQL

MyBatis #{} 与 \${} 的区别？

#{} 使用预编译参数，能避免大部分 SQL 注入；\${} 是字符串替换，只能用于严格白名单控制的动态标识符。

MyBatis 一级缓存与二级缓存？

一级缓存是 SqlSession 级默认缓存；二级缓存是 Mapper namespace 级，需要显式配置。业务中更应关注事务边界和缓存一致性。

如何避免 N+1 查询？

使用 join、批量 IN、一次查询后内存组装或合理的结果映射，结合实际数据量和分页；不要在循环中逐条查数据库。

联合索引最左前缀是什么？

B+Tree 联合索引按列顺序排序，查询通常从最左列开始才能充分利用；范围条件后的列用于进一步定位的能力会受影响。

覆盖索引是什么？

查询所需列都能从索引取得，无需回表。可减少 I/O，但索引过宽会增加存储与写入成本。

MySQL 事务隔离级别？

读未提交、读已提交、可重复读、串行化。InnoDB 默认常见为可重复读，并通过 MVCC 与锁处理并发。

MVCC 是什么？

通过版本链和 Read View 让读操作看到符合隔离级别的快照，减少读写阻塞；当前读和写仍可能加锁。

慢 SQL 如何排查？

先确认请求和 SQL，查看慢查询日志与 EXPLAIN，检查扫描行数、索引选择、排序/临时表、返回列和参数分布，再用真实数据验证优化。

分页为什么越往后越慢？

LIMIT offset 需要扫描并丢弃前面大量行。可用基于稳定排序键的 seek/keyset 分页，或先走覆盖索引取 ID 再回表。

第十二部分 Redis、RabbitMQ 与微服务基础

12.1 Redis

Redis 常见数据结构？

String、Hash、List、Set、Sorted Set、Stream 等。要根据访问模式选结构，不是把 Redis 只当 String 缓存。

缓存穿透、击穿、雪崩？

穿透是查不存在数据，可用空值/布隆过滤器；击穿是热点 key 失效并发回源，可用互斥重建/逻辑过期；雪崩是大量 key 同时失效或 Redis 故障，可加随机 TTL、多级缓存和限流。

缓存一致性怎么做？

常见 cache-aside：先更新数据库，再删除缓存；结合重试、消息或订阅 binlog 提高最终一致性。不能把更新缓存和数据库误称为天然原子。

Redis 分布式锁的注意点？

设置过期时间、唯一 value、只释放自己的锁、考虑任务超时和续期。Redis 单点/网络分区下的强一致语义需谨慎评估。

本项目 Redis 承担哪些不同角色？

Agent checkpoint、会话锁、归档 Stream；Java FAQ 缓存使用独立配置。RAG 向量索引不写 Redis。

12.2 RabbitMQ 与消息队列

回答边界：简历写的是‘了解 RabbitMQ，有一定使用经验’，项目当前归档通道仍是 Redis Stream。可以讲消息队列原理和迁移思路，不要说项目已经使用 RabbitMQ。

为什么使用消息队列？

解耦生产与消费、削峰填谷、异步处理和失败重试。代价是最终一致性、重复消息、顺序和运维复杂度。

RabbitMQ 的 exchange 类型？

direct 按 routing key 精确匹配；topic 按模式匹配；fanout 广播；headers 按消息头。

如何保证消息不丢？

生产端 publisher confirm、交换机/队列/消息持久化、消费者手动 ACK、失败重试和死信队列；仍需端到端幂等。

重复消费怎么处理？

MQ 常提供至少一次语义。消费者以业务唯一键去重，或让数据库写入具备唯一约束/幂等覆盖。

Redis Stream 与 RabbitMQ 怎么选？

Redis Stream 适合已有 Redis、吞吐适中和轻量事件流；RabbitMQ 路由、确认、死信和运维工具更成熟。项目可将归档通道迁移到 RabbitMQ，但不是简单替换 API。

死信队列有什么用？

保存多次失败、过期或被拒绝的消息，便于隔离、告警和人工处理，避免坏消息阻塞主队列。

第十三部分 简历技术栈追问与诚实回答边界

13.1 可以主讲、可以补充、只能了解

层级	技术	建议表达
主讲	LangChain、LangGraph、RAG、FastAPI、Chroma	结合项目节点、代码与失败策略回答
主讲	Python 基础、异步、Pydantic、SSE	能解释为何使用及常见坑
补充	Java、Spring Boot、MyBatis、MySQL	有 CRUD 和接口集成经历，按真实深度回答
补充	Redis	项目中确有 checkpoint、锁和 Stream；区分不同 Redis 用途

层级	技术	建议表达
了解	RabbitMQ、MCP	讲概念、适用场景和未来方案，不声称当前项目已落地
了解	CNN、RNN、Attention、Transformer	论文排除，只准备基础原理，不抢主线
非主项	HTML/CSS/JS、Vue/React	页面主要由 AI 辅助，只说明接口联调

13.2 易被追问的简历措辞

简历写 ‘熟悉 FastAPI’ 还是 ‘了解 FastAPI’ ？

如果能独立解释应用工厂、lifespan、依赖、Pydantic、StreamingResponse、断连和测试，可以写 ‘熟悉并用于项目’ ；若只是运行过接口，保留 ‘了解’ 更稳妥。当前代码足以支持重点准备后提升表述。

‘设计受控问答编排流程’ 如何证明？

说清节点、状态、条件边、默认 RAG、查询改写降级、索引失败禁用旧库、输出校验和 checkpoint，而不是只说用了 LangGraph。

‘负责向量知识库开发’ 如何证明？

能解释真相源、清单、fingerprint、Loader、分块参数、Embedding 兼容问题、Chroma 元数据、TopK、阈值、删除与重建。

‘参与后台管理系统后端研发’ 如何回答？

说明自己完成过哪些 CRUD、参数校验、Mapper/SQL 和表结构联调；如果没有独立负责权限、事务或性能优化，不要扩张。

MCP 被问到怎么办？

先定义 MCP 是模型与工具/资源之间的标准协议，再说当前项目未接入 MCP，因为 RAG 是确定性内部流程；如果未来接入，会关注权限、schema、超时和审计。

前端被追问怎么办？

直接说明页面由 AI 辅助生成，自己主要定义接口和联调。能讲清 POST SSE、事件格式和错误恢复，但不把 Vue 响应式、组件设计说成个人强项。

如果面试官问项目代码是否全部自己写？

诚实说明使用 Codex 等 AI 编程工具辅助，但架构约束、接口契约、测试标准和问题排查由自己理解并验证；对无法解释的代码不声称掌握。

如果问到没做过的生产压测？

不要编数字。可以说尚未做系统压测，并给出计划：并发模型、首 token/P95、错误率、连接数、模型限流、Redis/Chroma 指标和瓶颈定位。

13.3 推荐的项目职责表述

可直接用于简历/面试：基于 FastAPI 构建独立 Agent 微服务，使用 LangGraph 编排输入规范化、问题路由、查询改写、RAG 检索、流式生成和输出校验；基于 Java 发布清单实现维修手册解析、分块、Embedding 与 Chroma 增量索引，并通过 SSE 向 Java 服务输出状态、引用与增量文本；使用 Redis checkpoint、会话锁和 Redis Stream 维护热上下文及异步归档，支持 MongoDB 冷恢复。

第十四部分 模拟面试、复习计划与自测清单

14.1 一轮模拟面试题

1. 请用 90 秒介绍智能客服项目，并明确你的个人职责。
2. 为什么拆成 Java 与 Python 两个服务？数据边界怎么划分？
3. FastAPI lifespan 中初始化了哪些资源？为什么不在每次请求创建？
4. SSE 建立后模型异常，为什么不能返回 HTTP 500？
5. LangGraph 的 State、Node、Edge、checkpointer 分别是什么？
6. DIRECT/RAG 路由失败为什么默认 RAG？
7. 查询改写、索引同步和检索分别如何降级？

- 8. 如何判断一份手册是否需要重新向量化？
- 9. Chroma 中保存哪些元数据？引用如何与答案对应？
- 10. Redis checkpoint 丢失后如何从 MongoDB 恢复？
- 11. 同一会话并发请求会发生什么？你如何处理？
- 12. Redis Stream 消费者崩溃后 Pending 消息怎么恢复？
- 13. Java 最终为什么要在 MySQL 提交后才向浏览器发 done？
- 14. 项目中 RabbitMQ 是否已使用？如果迁移要改哪些语义？
- 15. 前端由 AI 辅助生成，你如何证明自己理解整体系统？
- 16. 当前项目离生产还有哪三项最重要的差距？

14.2 回答结构

- 先给结论：一句话说明选择。
- 再给项目证据：具体文件、节点、字段或数据流。
- 解释取舍：收益、成本和替代方案。
- 承认边界：哪些已经实现，哪些只是下一步。

14.3 14 天复习计划

天数	主题	输出要求
D1	项目架构与 90 秒介绍	不看稿讲两遍并录音
D2-D3	FastAPI + Pydantic + SSE	能手写最小流式接口和错误事件
D4-D5	Python async/await 与并发	解释事件循环、to_thread、锁和取消
D6-D7	LangChain / LangGraph	白纸画状态图并讲每个失败分支
D8-D9	RAG / Chroma	讲清增量索引与评测指标
D10	Redis / Mongo / Stream	画出热状态、冷恢复和归档时序
D11-D12	Java / Spring / MyBatis / MySQL	完成基础八股口述
D13	RabbitMQ、MCP、生产化	只讲边界与方案，不越界
D14	完整模拟面试	45 分钟，复盘含糊回答并补代码证据

14.4 最终自测

- 我能不看稿讲清 Java 与 Python 的职责边界。
- 我能画出 LangGraph 的全部节点和条件边。
- 我能解释为何旧 Chroma 索引在清单失败时不能继续使用。
- 我能区分 HTTP 错误与 SSE error 终态。
- 我能解释 Redis checkpoint、Stream、Lock 三种用途互不相同。
- 我能解释当前 RabbitMQ、MCP 和前端的真实掌握边界。
- 我能说出至少五项生产化缺口及改进顺序。

附录 官方资料与项目代码复习路径

A.1 官方资料

- [FastAPI - Lifespan Events](#)
- [FastAPI - Dependencies](#)
- [FastAPI - StreamingResponse](#)
- [FastAPI - Server-Sent Events](#)
- [Pydantic - Validators](#)
- [Python - asyncio](#)
- [LangGraph - Graph API](#)
- [LangGraph - Persistence](#)
- [LangGraph - Streaming](#)
- [LangChain - Retrieval](#)
- [Chroma - Query and Get](#)
- [Redis - Streams](#)
- [Redis - XAUTOCLAIM](#)
- [Spring - Transaction Management](#)

- [MyBatis - Mapper XML](#)
- [MySQL - InnoDB Transaction Model](#)
- [RabbitMQ - Tutorials](#)

A.2 项目代码复习顺序

1. agent_service/src/agent_service/main.py: 应用工厂、lifespan 和资源创建。
2. agent_service/src/agent_service/api/routes/chat.py: SSE、断连和错误映射。
3. agent_service/src/agent_service/schemas/chat.py: Pydantic 契约和 camelCase。
4. agent_service/src/agent_service/graph/state.py: 长期状态与运行上下文边界。
5. agent_service/src/agent_service/graph/workflow.py: 全部 LangGraph 节点与分支。
6. agent_service/src/agent_service/services/agent_runtime.py: 锁、恢复、运行和归档。
7. agent_service/src/agent_service/services/manual_rag.py: 索引同步、解析、分块和检索。
8. agent_service/src/agent_service/models/gateway.py: Mock/真实模型适配。
9. agent_service/src/agent_service/services/context_archive.py: Redis Stream 与冷恢复。
10. agent_service/src/agent_service/workers/context_archive_worker.py: 消费、ACK 和 XAUTOCLAIM。
11. agent_service/tests: 按测试名复盘每个边界条件。
12. xc_agent: 只复习 Java 网关、事务落库、MyBatis 和 MySQL, 不把前端作为主线。

最后提醒: 面试中最有说服力的不是术语数量, 而是你能把一个节点、一条事件或一个失败分支讲到代码级别, 并清楚说出它为什么存在。

— 完 —